

Горчак Дмитрий, 212. Библиотека STL: контейнеры, итераторы, алгоритмы. Двухнаправленный список и упорядочивание объектов

STL (Standard Template Library – библиотека стандартных шаблонов) – часть стандартной библиотеки C++, в которой лежат готовые структуры данных и алгоритмы: списки, массивы, словари, сортировка, поиск. Их не нужно писать самому – достаточно подставить свой тип данных.

В первой части разобрано, из чего состоит STL и какие в ней контейнеры. Во второй – как сложить объекты в двухнаправленный список и упорядочить его по разным признакам. Примеры взяты из учебного проекта cones: программа читает из файла параметры фигур и строит точку, круг или конус в зависимости от того, сколько чисел в строке (два, три или четыре). Фигуры нужно расположить сначала по возрастанию площади основания, потом по возрастанию объёма.

Из чего состоит STL

- 1) контейнеры (container – хранилище) – объекты, которые хранят другие объекты: vector, list, deque, set, map;
- 2) итераторы (iterator – перебирающий) – единый способ проходить содержимое любого контейнера;
- 3) алгоритмы – готовые операции: сортировка, поиск, подсчёт, копирование, преобразование;
- 4) функциональные объекты и лямбда-функции – ими задают правило сравнения или условие отбора и передают алгоритму;
- 5) адаптеры (stack, queue, priority_queue) и распределители памяти.

Шаблон (template – образец) означает, что тип хранимых данных не задан заранее, а подставляется при использовании: `std::list<int>` – список чисел, `std::list<Cone>` – список конусов. Код контейнера написан один раз и работает с любым типом.

Контейнеры ничего не знают об алгоритмах, а алгоритмы – о контейнерах; связывают их итераторы. Поэтому сортировка написана один раз и работает с разными контейнерами.

Контейнеры

Для каждой операции в стандарте указано, как растёт время её работы с увеличением числа элементов n : линейно (пропорционально n), за $\log n$ (почти не растёт) или за постоянное время. Выбор контейнера – это выбор того, какие операции окажутся дешёвыми.

Контейнер	Как устроен	Дёшево	Дорого
vector	динамический массив	доступ по номеру, добавление в конец	вставка и удаление в середине
list	двусвязный список	вставка и удаление в любом месте	нет доступа по номеру, поиск перебором
forward_list	односвязный список	минимум памяти на элемент	проход только вперёд
deque	блоки элементов	добавление с обоих концов	вставка в середину

set, map	сбалансированное дерево	поиск и вставка за $\log n$, лишняя память на связи порядок
unordered_set, map	хеш-таблица	поиск за постоянное время элементы не упорядочены

Адаптер (adapter – переходник) – не самостоятельное хранилище, а надстройка над контейнером, которая оставляет только нужные операции: stack (стек – кладём и снимаем с одного конца), queue (очередь – кладём в конец, берём из начала), priority_queue (первым выдаётся наибольший элемент).

Итераторы

Итератор – объект, который указывает на элемент контейнера и умеет переходить к соседнему. Его разыменовывают (*it – получить сам элемент), сдвигают (++it) и сравнивают с другим итератором.

Диапазон задаётся парой итераторов [first, last): last указывает уже за последний элемент, поэтому цикл заканчивается по условию `it != end()`, а не `it <= end()`.

Итераторы различаются тем, что умеют. У вектора итератор произвольного доступа (random access): к нему можно прибавить число и сразу попасть на нужный элемент. У списка итератор двунаправленный (bidirectional): он умеет шагать только вперёд и назад, по одному элементу. Отсюда следствие: общий алгоритм `std::sort` со списком работать не будет.

Алгоритмы

Готовые операции лежат в заголовочных файлах `<algorithm>` и `<numeric>`: `sort` и `stable_sort` (упорядочить), `find_if` (найти первый подходящий элемент), `count_if` (сосчитать подходящие), `max_element` и `min_element` (найти крайний по заданному правилу), `accumulate` (накопить сумму), `remove_if` (убрать подходящие).

Правило сравнения или отбора передаётся отдельным аргументом, поэтому одна и та же сортировка упорядочивает и по площади, и по объёму.

Двунаправленный список: устройство

Двунаправленный (двусвязный) список состоит из заголовка и элементов. Заголовок хранит указатель на первый элемент и число элементов `n`. Каждый элемент хранит три вещи: сам объект, ссылку на предыдущий элемент и ссылку на следующий.

[заголовок: $n = 3$, first] --> [Cone A] <--> [Cone B] <--> [Cone C]

Из такого устройства следуют все свойства списка:

- 1) по списку можно идти в обе стороны – от начала и от конца;
- 2) вставка и удаление сводятся к переключению двух ссылок: соседние элементы не сдвигаются и не копируются;
- 3) обращения по номеру нет – чтобы добраться до пятого элемента, нужно пройти через четыре предыдущих;
- 4) на каждый элемент тратится дополнительная память под две ссылки.

В стандартном `std::list` список замкнут в кольцо через служебный элемент, а число элементов хранится в заголовке, поэтому `size()` выполняется мгновенно. Адреса объектов при вставках, удалениях и сортировке не меняются — переставляются только ссылки.

Создание списка и добавление объектов

Нужен заголовочный файл `<list>`. Список объявляется одной строкой, а объекты добавляются по мере создания. В одном списке лежат разные фигуры, поэтому хранятся не сами объекты, а указатели на общий базовый класс `Point` — так работает полиморфизм:

```
#include <list>
#include <memory>

typedef std::shared_ptr<Point> FigurePtr; /* указатель на любую фигуру */

std::list<FigurePtr> figures;

for (std::size_t i = 0; i < loaded.size(); ++i) {
    figures.push_back(loaded[i]); /* новый элемент в конец списка */
}
```

`push_back` добавляет в конец, `push_front` — в начало, `front` и `back` возвращают первый и последний элементы, `size` — число элементов, `empty` — пуст ли список.

Перебор идёт итератором; записи вида `cones[3]` у списка нет:

```
for (std::list<FigurePtr>::const_iterator it = figures.begin();
     it != figures.end(); ++it) {
    (*it)->print(std::cout); /* у каждой фигуры свой вывод */
}
```

`const_iterator` — итератор, через который элементы можно только читать. Звёздочка даёт сам элемент списка (указатель на фигуру), а стрелка вызывает метод объекта, на который он указывает.

Упорядочивание по площади и по объёму

Порядок задаётся функцией сравнения: она получает два элемента и возвращает `true`, если первый должен стоять раньше. Площадь и объём не хранятся в полях, а считаются методами; у точки площади нет, у точки и круга нет объёма, поэтому для них возвращается ноль:

```
bool less_by_area(const FigurePtr &left, const FigurePtr &right) {
    return area_of(left) < area_of(right);
}

bool less_by_volume(const FigurePtr &left, const FigurePtr &right) {
    return volume_of(left) < volume_of(right);
}

figures.sort(less_by_area); /* порядок по площади основания */
figures.sort(less_by_volume); /* порядок по объёму */
```

Вызывается не общий алгоритм `std::sort`, а собственный метод списка `sort`. Причина в итераторах: `std::sort` прыгает по элементам туда-сюда, а итератор списка так не умеет. Метод `list::sort` реализован сортировкой

слиянием: он переставляет ссылки между элементами, не копируя сами объекты, работает за $n \log n$ и сохраняет взаимный порядок равных элементов (такую сортировку называют устойчивой).

Если список уже упорядочен, новый элемент можно сразу поставить на своё место. Поиск места – проход по списку, сама вставка стоит двух переключений ссылок независимо от длины списка:

```
void insert_sorted_by_volume(std::list<FigurePtr> &figures, const FigurePtr &value) {
    std::list<FigurePtr>::iterator place = figures.begin();

    while (place != figures.end() && volume_of(*place) < volume_of(value)) {
        ++place;
    }
    figures.insert(place, value); /* insert ставит элемент перед place */
}
```

Удаление по условию делается одной командой: `remove_if` проходит список и выбрасывает подходящие элементы, не трогая остальные.

```
figures.remove_if(small_volume); /* убрать фигуры с объёмом меньше порога */
```

Результат

Два порядка получаются разными. Заметнее всего это на конусе с широким, но низким основанием ($r = 2.80$, $h = 2.00$): по площади основания он предпоследний, а по объёму – только тринадцатый из восемнадцати. Круги в порядке по площади стоят наравне с конусами, а в порядке по объёму уходят в начало списка вместе с точками, потому что объёма у них нет.

--- Sorted by base area ---

14	Circle	12.50	17.50	2.00	-	12.57	-
15	Cone	19.00	4.00	2.20	9.00	15.21	45.62
16	Cone	14.00	12.00	2.40	8.00	18.10	48.25
17	Cone	3.00	5.50	2.80	2.00	24.63	16.42
18	Cone	9.00	8.00	3.00	10.00	28.27	94.25

--- Sorted by volume ---

13	Cone	3.00	5.50	2.80	2.00	24.63	16.42
14	Cone	8.00	17.00	1.70	5.50	9.08	16.65
15	Cone	15.00	15.50	1.80	6.50	10.18	22.05
16	Cone	19.00	4.00	2.20	9.00	15.21	45.62
17	Cone	14.00	12.00	2.40	8.00	18.10	48.25
18	Cone	9.00	8.00	3.00	10.00	28.27	94.25

Двунаправленность проверяется обратным проходом: тот же список читается от конца к началу обратным итератором (`rbegin` и `rend`), и объёмы идут по убыванию.

```
for (std::list<FigurePtr>::const_reverse_iterator it = figures.rbegin();
     it != figures.rend(); ++it) {
    std::cout << volume_of(*it) << ' ';
}
```

Список или вектор

Список выгоден, когда элементы добавляются и удаляются по одному, а доступ к ним последовательный: объекты создаются при чтении файла и потом упорядочиваются целиком. Вектор выгоден, когда нужен доступ по номеру и

важна скорость перебора: его элементы лежат в памяти подряд, и процессор читает их быстрее.

Часто в одной задаче нужны оба: например, матрица попарных расстояний n на n – это вектор векторов с доступом по номерам, а очередь точек при обходе графа – адаптер queue.

Заключение

Готовый шаблон списка избавляет от написания своей структуры данных: достаточно подставить нужный тип и задать правила сравнения. Понимать устройство всё равно нужно – именно из него следует, почему у списка своя сортировка, почему нет обращения по номеру и почему вставка в середину дешевле, чем в векторе.

*[объект создан] -(push_back)-> [элемент списка: объект + две ссылки]
 [правило сравнения] -(list::sort)-> [порядок по площади или по объёму]
 [итератор] -(++ / --)-> [проход вперёд и назад]*

Приложение. Памятка по `std::list`

Цель	Операция	Пример	Описание
Объявить список	<code>std::list<T></code>	<code>std::list<FigurePtr> figures;</code>	T – тип хранимых элементов
Добавить в конец	<code>push_back</code>	<code>figures.push_back(figure) ;</code>	новый элемент становится последним
Добавить в начало	<code>push_front</code>	<code>figures.push_front(figure);</code>	новый элемент становится первым
Вставить в середину	<code>insert</code>	<code>figures.insert(place, figure);</code>	ставит элемент перед итератором place
Удалить элемент	<code>erase</code>	<code>figures.erase(place);</code>	убирает элемент, на который указывает place
Удалить по условию	<code>remove_if</code>	<code>figures.remove_if(small_v olume);</code>	убирает все подходящие элементы
Первый и последний	<code>front, back</code>	<code>figures.front()->print(ou t);</code>	возвращают сам элемент, а не итератор
Число элементов	<code>size, empty</code>	<code>figures.size();</code>	size выполняется мгновенно
Проход вперёд	<code>begin, end</code>	<code>for (it = figures.begin(); ...)</code>	end указывает за последний элемент
Проход назад	<code>rbegin, rend</code>	<code>for (it = figures.rbegin(); ...)</code>	обратный итератор, от конца к началу
Упорядочить	<code>sort</code>	<code>figures.sort(less_by_area);</code>	сортировка слиянием, $n \log n$, устойчивая
Перевернуть	<code>reverse</code>	<code>figures.reverse();</code>	меняет порядок элементов на обратный
Объединить списки	<code>merge, splice</code>	<code>a.merge(b);</code>	переносит элементы без копирования
Очистить	<code>clear</code>	<code>figures.clear();</code>	удаляет все элементы